

Design of Loss function

Yi Cao¹

¹Department of Financial and Actuarial Mathematics, Xi'an Jiaotong-Liverpool University, Suzhou,
Jiangsu 215123, PR China.

October 9, 2025

JEL code: C63, F47

Keywords: machine learning

1 Brainstorm

The goal is to create a differentiable loss that penalises sign mismatches heavily (e.g., predicted positive return when true is negative, or vice versa) while being more lenient on magnitude errors with the same sign. This contrasts with symmetric losses like Mean Squared Error (MSE: $\frac{1}{n} \sum (y - \hat{y})^2$) or Root MSE (RMSE: $\sqrt{\text{MSE}}$), which treat overestimation and underestimation equally regardless of sign.

2 Rationale for the Loss Function Design

In asset return prediction (e.g., stock returns), errors aren't just about magnitude; direction matters critically due to trading implications:

- **Sign Mismatch Penalty:** If true return $y > 0$ (bullish) but predicted $\hat{y} < 0$ (bearish), this could lead to wrong-way trades (e.g., shorting a rising asset), amplifying losses. Similarly for the reverse. A high penalty here aligns with behavioural finance (e.g., prospect theory's loss aversion, Kahneman & Tversky, 1979) and risk management, where directional accuracy is key for strategies like momentum or arbitrage.
- **Asymmetric Treatment:** Same-sign errors (e.g., both positive but predicted too high/low) are less catastrophic, as they might still inform correct buy/sell decisions, just with suboptimal sizing. Thus, the loss should be "flatter" for positive errors ($X = \hat{y} - y > 0$, overestimation) to encourage models that get direction right, even if magnitude is off.
- **Curve Requirements:** Based on the discussions above, the loss $L(X)$ should be U-shaped but asymmetric: steep dip left of 0, flatter rise right:
 - Be minimum near $X=0$ (accurate prediction).
 - Rise sharply for $X < 0$ (severe penalty for underestimation or sign flips).
 - Rise gradually for $X > 0$ (milder penalty for overestimation).
 - Be differentiable (for gradient-based optimisation in NNs like backpropagation).
 - Non-negative, convex for stability, and scalable (e.g., via hyperparameters).

Why Not Standard Losses?

- MSE/RMSE: Symmetric, no sign sensitivity ($L(X) = X^2$ treats +e and -e equally).

- Alternatives like MAE (absolute error) are symmetric; Huber is robust to outliers but not sign-asymmetric.
- Quantile loss or pinball loss (for quantile regression) can be asymmetric but focus on tails, not signs.
- We need a custom design, inspired by activations (e.g., sigmoid for smoothness) but adapted for loss (positive, minimum at 0).

Design Principles:

- **Asymmetry:** Use piecewise-inspired functions (e.g., steeper exponential decay for $X < 0$, softer quadratic for $X > 0$), joined smoothly (e.g., via softplus for differentiability).
- **Differentiability:** Ensure continuous first derivative (e.g., avoid hard ReLU-like kinks).
- **Inspiration from Activations:** Sigmoid ($\sigma(x) = 1/(1 + e^{-x})$) for S-shapes; tanh for bounded symmetry; softplus ($\ln(1 + e^x)$) for smooth positives; combine with quadratics or exponentials.
- **Sign Sensitivity:** Incorporate a term that amplifies when $\text{sign}(\hat{y}) \neq \text{sign}(y)$, but since $X = \hat{y} - y$, we can embed via hyperbolic or logistic forms that “punish” negative X more.
- **Scalability:** Add parameters for tuning sharpness/flatness (e.g., for different assets’ volatility).

3 Understanding the Loss Function MADL/GMADL and Its Issues

3.1 MADL/GMADL ideas

MADL and GMADL loss functions are from the papers Michańków et al. (2024b,a).

We first review their ideas and potential issues and then explore the way to improve them.

Core Idea: MADL/GMADL simulates “investment outcomes” by penalizing wrong directional predictions (opposite signs of r_i and $\hat{r}_i$) with positive loss ($+|r_i|$ or scaled equivalent) and “rewarding” correct directions with negative loss ($-|r_i|$ or scaled).

The smooth GMADL version uses a sigmoid approximation for differentiability (with $a=100$ for sharpness) and $|r_i|^b$ ($b=2$ for squaring, emphasizing larger returns).

Key Mechanism:

- The sign term (or its smooth approx) acts as a directional multiplier: same direction $\rightarrow$ negative contribution (profit-like);
- opposite $\rightarrow$ positive (loss-like). Multiplied by $|r_i|^b$, it weights by observed return magnitude.

The Problematic Situation: in the example of true return is $r_i=0.01$:

- Opposite direction ($\hat{r}_i=-0.1$): Loss $\approx -3.1\text{e-}5$ (slightly negative).
- Perfect match ($\hat{r}_i=0.01$): Loss $\approx -7.1\text{e-}5$ (more negative).
- Over-prediction but same direction ($\hat{r}_i=0.1$): Loss $\approx -1.15\text{e-}4$ (even more negative).

This is counterintuitive because:

- The loss becomes more negative (i.e., “better“ in a numerical sense) for predictions with larger $|\hat{r}_i|$ in the correct direction, even if the magnitude is wrong (over-predicting is “rewarded” more than perfect prediction).
- In ML optimization (which minimizes loss), this incentivizes exaggerated predictions in the correct direction, ignoring magnitude accuracy.
- For small r_i , the smooth sigmoid doesn’t fully approximate the sharp Sign function, leading to small negative losses even for wrong directions (due to the -0.5 to +0.5 range scaled by small $|r_i|^b$).
- Broader Issues: The loss can be negative overall, which is unstable for optimizers (e.g., gradient descent assumes non-negative losses); it doesn’t penalize magnitude errors (e.g., $\hat{r}_i=100$ vs. $r_i=0.01$ in correct direction yields huge “reward”); and it’s not bounded below, risking divergence.

The function encourages directional accuracy but fails as a proper loss due to its reward structure and lack of magnitude penalty.

3.2 Ways to Improve the Loss Function

Here are several concrete improvement ideas, ranging from simple tweaks to more advanced reformulations. Each includes rationale, a revised formula, pros/cons, and implementation notes. These make the loss non-negative, minimization-friendly, differentiable, and more balanced for direction + magnitude.

3.2.1 Flip the Sign and Add Magnitude Penalty (Basic Fix for Inversion and Accuracy)

Rationale

Reverse the negative “rewards” to positive penalties, ensuring wrong directions/magnitudes yield higher loss. Add a term for $|r_i - \hat{r}_i|$ to penalize over/under-prediction, balancing direction with precision. This keeps the “investment simulation” flavour but aligns with standard loss minimization. Revised Formula is as:

$$\text{Improved MADL} = \frac{1}{N} \sum_{i=1}^N \left[(1 - \text{Sign}(r_i \cdot \hat{r}_i)) \cdot |r_i|^b + c \cdot |r_i - \hat{r}_i|^d \right] \quad (1)$$

If a smooth version is needed, we may replace the function *Sign* with the sigmoid function $\text{Sigmoid} = \frac{1}{1 + \exp(-X)}$. Parameters may be $b=2$, $c=1$ (weight), $d=1$ or 2 ($L1$ or $L2$ norm for error):

$$\text{Improved MADL} = \frac{1}{N} \sum_{i=1}^N \left[\left(1 - \frac{1}{1 + \exp(-a \cdot r_i \cdot \hat{r}_i)} \right) \cdot |r_i|^b + c \cdot |r_i - \hat{r}_i|^d \right] \quad (2)$$

When $d = 2$, the latter part is MSE.

- Simple structure
- If prediction $\hat{r}_i$ has correct direction as original return $r_i \rightarrow$ low loss error term;
- If prediction $\hat{r}_i$ has wrong direction $\rightarrow$ high loss
- It encourages both direction and magnitude accuracy

Cons: May need tuning c/d to balance terms.

3.2.2 Make It a Directional Huber Loss (Robust to Outliers and Small Values)

Rationale: Build on Huber loss (robust to outliers, smoother than MAE/MSE) but incorporate directional penalty. For small r_i , Huber reduces sensitivity. Penalize wrong directions multiplicatively, while using Huber on the signed error to handle the “profit/loss” robustly. The revised Formula:

$$\text{Directional Huber} = \frac{1}{N} \sum_{i=1}^N [\text{pen}(r_i, \hat{r}_i) \cdot H_\delta(r_i - \hat{r}_i)]$$

$$\begin{aligned}
H_\delta(e) &= \begin{cases} \frac{1}{2}e^2 & |e| \leq \delta \\ \delta(|e| - \frac{1}{2}\delta) & |e| > \delta \end{cases} \\
\text{pen} &= \frac{1}{2}(1 - \tanh(a \cdot r_i \cdot \hat{r}_i))
\end{aligned} \tag{3}$$

where tanh output is in $[-1,1]$. Parameters: $\delta=1$ (threshold), $a=10$ (less sharp than 100 for stability), optionally multiply by $|r_i|^b$.

Pros:

- Robust to extreme returns (e.g., volatility);
- differentiable;
- fixes small r_i issues by capping large errors;
- economically interpretable as “risk-adjusted loss.”

Cons:

- Slightly more complex;
- We have to tune the δ based on return scales.

3.2.3 Incorporate Asymmetric Penalties with Quantile Loss (For Risk-Averse Optimization)

Rationale: In finance, wrong directions (e.g., predicting up when down) are costlier than magnitude errors. Use quantile loss (pinball loss) weighted by the direction term, penalizing under/over-predictions asymmetrically. This improves on the function by linking to economic concepts like Value-at-Risk (VaR), where missing downside risks (negative returns) is penalized more. The revised formula is as:

$$\begin{aligned}
\text{Refined QDL} &= \frac{1}{N} \sum_{i=1}^N [(1 - \text{Sign}(r_i \cdot \hat{r}_i)) \cdot \rho_\tau(r_i - \hat{r}_i)] + \lambda \cdot |r_i - \hat{r}_i| \\
\rho_\tau &= \tau \cdot \max(e, 0) + (1 - \tau) \cdot \max(-e, 0) \\
e &= r_i - \hat{r}_i
\end{aligned} \tag{4}$$

$\rho_\tau(e)$ is the quantile loss function for error with $\tau \in (0, 1)$ controlling asymmetry (e.g., $\tau=0.5$ for symmetric MAE, or $\tau=0.1$ for penalizes positive errors more, i.e., underpredictions).

For smoothness (as in your GMADL), replace Sign with a sigmoid approximation: $\text{Sign} \approx \frac{2}{1+e^{-a \cdot r_i \cdot \hat{r}_i}} - 1$ (outputs $[-1,1]$; $a=10$ for gradual slope).

$$\begin{aligned} \text{Refined QDL} &= \frac{1}{N} \sum_{i=1}^N \left[2 \cdot \left(1 - \frac{1}{1 + e^{-a \cdot r_i \cdot \hat{r}_i}} \right) \cdot \rho_\tau(r_i - \hat{r}_i) \right] + \lambda \cdot |r_i - \hat{r}_i| \\ \rho_\tau &= \tau \cdot \max(e, 0) + (1 - \tau) \cdot \max(-e, 0) \\ e &= r_i - \hat{r}_i \end{aligned} \tag{5}$$

Assume $\tau=0.5$ (symmetric), $\lambda=0.5$, and three examples:

- Example 1: $r_i=0.01$, $\hat{r}_i=0.02$ (same direction, small error): $\text{Sign}=1$, $\text{multiplier}=1-1=0$; quantile $\rho=0.5|0.01|=0.005$; $\text{magnitude}=0.5|-0.01|=0.005$; Total is around 0.01.
- Example 2: $r_i=0.01$, $\hat{r}_i=-0.1$ (opposite direction): $\text{Sign}=-1$, $\text{multiplier}=1-(-1)=2$; $\rho=0.5|0.11|=0.055$; $\text{magnitude}=0.5 \cdot 0.11=0.055$; Total is around 0.22 (high penalty).
- Example 3: $r_i=0$ (neutral): $\text{Sign}=0$, $\text{multiplier}=1-0=1$; $\rho=0.5|0 - \hat{r}_i|$ (depends on $\hat{r}_i$); plus magnitude term.

Pros: Handles imbalance naturally (focuses on tails); economically ties to risk management (e.g., penalizes missing depegging losses); differentiable.

Cons: Requires choosing based on domain (e.g., risk aversion).

3.2.4 Add Regularization and Bound the Loss (For Stability in Optimization)

Rationale: Correct predictions with large $\hat{r}_i$ in the same direction can make the loss highly negative (approaching $-\infty$ if $\hat{r}_i$ grows. The loss encourages over-prediction in the correct direction, ignoring magnitude accuracy. This leads to issues like non-convex landscapes, sensitivity to small r_i and poor handling of edge cases $r_i=0$.

The fix: Bound the loss to $[0, -\infty)$ by clipping negative contributions to 0 (turning “rewards” into zero penalty, focusing only on errors), and add regularization (e.g., L2 penalty on the prediction error) to discourage wild exaggerations while enforcing magnitude closeness. This transforms it into a proper loss: minimized at 0 for perfect predictions, positive otherwise, and stable for training.

Economically, this interprets as: No penalty for spot-on predictions (ideal “zero-risk” trade), but increasing costs for directional errors or over-optimistic magnitudes, aligning with risk-averse

strategies in finance (e.g., avoiding over-leveraged bets). This makes it a proper loss for gradient descent. Revised Formula:

$$\text{Bounded GMADL} = \frac{1}{N} \sum_{i=1}^N \left[\max \left(0, - \left(\frac{1}{1 + e^{-ar_i \hat{r}_i}} - 0.5 \right) \cdot |r_i|^b \right) + \lambda \cdot |\hat{r}_i - r_i|^2 \right] \quad (6)$$

where the directional term $\left(\frac{1}{1 + e^{-ar_i \hat{r}_i}} - 0.5 \right)$ output $[-0.5, +0.5]$. Multiplied by -1 flips it to positive penalty for opposite prediction direction, and negative “reward” on same direction prediction. Bounding with $\text{Max}(0, \dots)$ clips negative values to 0. This ensures the directional part only contributes positively (penalties) or zero (perfect/no error), preventing $-\infty$ issues.

Pros: Ensures non-negative, convex-ish loss; stabilizes training; fixes your example (wrong direction $\rightarrow$ positive loss $>$ correct).

Cons: need to tune λ .

3.2.5 Hybrid with Ranking Loss (For Directional Focus in Finance)

Rationale: the original loss function emphasizes directional accuracy (same sign of r_i and $\hat{r}_i \rightarrow$ “reward”; but opposite direction $\rightarrow$ penalty).

In addition to this, we may frame this as a ranking problem: we rank predictions based on how well they align with the observed direction because in financial trading, pairwise orderings rather than absolute values are more important.

To do this, we add an additional pairwise ranking loss term. The ranking component compares pairs of predictions (e.g., for instances i and j) and penalizes if the model ranks a worse prediction higher. We define “worse” based on directional mismatch or large magnitude errors relative to the observed return.

For training, we create pairs (i, j) where $i \neq j$. To focus on meaningful comparisons:

- Positive pairs: Both have correct directions.
- Negative pairs: One correct, one wrong (penalize if wrong is ranked higher).
- Use sampling to avoid $O(N^2)$ complexity (e.g., random pairs per batch)

Then we can compute Ranking Loss: Use a pairwise loss, which models the probability that prediction i should rank higher than j based on their qualities or scores.

$$L_{\text{hybrid}} = (1 - \lambda) \cdot \text{GMADL} + \lambda \cdot L_{\text{rank}}$$

$$\begin{aligned}
\text{GMADL} &= \frac{1}{N} \sum_{i=1}^N \left[(-1) \cdot \left(\frac{1}{1 + e^{-a \cdot r_i \cdot \hat{r}_i}} - \frac{1}{2} \right) \cdot |r_i|^b \right] \\
L_{\text{rank}} &= \frac{1}{M} \sum_{(i,j)} [-\bar{P}_{ij} \log(P_{ij}) - (1 - \bar{P}_{ij}) \log(1 - P_{ij})] \\
P_{ij} &= \frac{1}{1 + e^{-(s_i - s_j)}} \\
s_k &= \text{Sign}(r_k \cdot \hat{r}_k) \cdot \exp \left(-\frac{|r_k - \hat{r}_k|}{|r_k| + \epsilon} \right) \\
&\quad (\epsilon \text{ small to avoid division by zero})
\end{aligned} \tag{7}$$

where P_{ij} is the probability that i ranks higher than j for pairs (i, j) . $\bar{P}_{ij}$ is the target probability: 1 if $r_i > r_j$ (i should rank higher), 0.5 if equal, 0 otherwise. For simplicity, set $\bar{P}_{ij} = 1$ if $s_i > s_j$ (based on ground truth), else 0.

s_k is a score for prediction k , e.g. this score is 1 for perfect prediction; -1 for large error. s_k combines directional correctness (Sign: binary good/bad) with precision (magnitude term: how close to truth). High s_k means the prediction is reliable (correct sign + small error), like a “high-confidence trade signal.” Low/negative s_k flags unreliability (wrong sign or huge error), like a “risky bet.”

s_k provides a scalar for comparison: a higher s_k leads to a better quality, so it should rank above lower ones. This creates an ordinal scale: The model learns to maximize differences in s_k for good vs. bad predictions, ensuring “quality” is optimized globally (not just per-sample).

4 Loss Function Comparison and Analysis

(this is complex, not easy to complete right now)

Traditional losses like Mean Squared Error (MSE) and Mean Absolute Error (MAE) are well-studied. New loss functions (e.g., for domain-specific tasks like financial return prediction or robust regression) requires not only empirical testing but also theoretical validation. This ensures properties such as convergence to global minima, smooth gradients for optimization, statistical efficiency, and resilience to noise/outliers.

Theoretically comparing new loss functions with traditional ones, e.g., MSE and MAE, focusing on convergence, first derivative properties, statistical attributes, and robustness, is crucial. We draw from optimization theory, statistical learning, and empirical risk minimization Vapnik (1999).

4.1 Convergence Properties

Convergence assesses if optimization (e.g., gradient descent) reliably finds minima. Key aspects:

- **Convexity:** A convex loss ensures a global minimum (no local traps). MSE is convex; MAE is convex but non-differentiable at 0.
- **Lipschitz Continuity:** Bounds gradient changes, aiding stable learning rates.
- **Analysis Method:** Check Hessian matrix (positive semi-definite for convexity) or subgradient for non-smooth losses.

4.2 First Derivative Properties

The gradient (first derivative) drives optimization; smoothness ensures stable updates.

- **Differentiability:** Smooth losses (e.g., MSE) have continuous gradients; piecewise (e.g., MAE) may cause zigzagging.
- **Gradient Vanishing/Exploding:** Analyze magnitude—e.g., sigmoid-based losses may vanish for large errors.
- **Analysis Method:** Compute $\frac{\partial L}{\partial \mathbf{y}}$; plot for behavior.

4.3 Robustness

Robustness measures resilience to outliers/noise.

- **Breakdown Point:** Fraction of corrupted data before failure (MAE: 50%; MSE: 0%).
- **Influence Function:** Sensitivity to single points.
- **Analysis Method:** M-estimators or simulation with contaminated data.

4.4 Framework for Theoretical Comparison

To compare a new loss $L_{\text{new}}(\mathbf{y}, \hat{\mathbf{y}})$ with MSE $L_{\text{MSE}} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$ and MAE $L_{\text{MAE}} = \frac{1}{n} \sum |y_i - \hat{y}_i|$:

4.4.1 Convexity

This is to show $L(\lambda \mathbf{z}_1 + (1 - \lambda) \mathbf{z}_2) \leq \lambda L(\mathbf{z}_1) + (1 - \lambda) L(\mathbf{z}_2)$

Purpose: Convexity ensures the loss surface has no local minima (only a global one), making optimization (e.g., gradient descent) reliable and guaranteeing convergence to the optimal solution. Non-convex losses can trap models in suboptimal points. Comparing to MSE (convex) or custom losses helps validate if new loss is as “optimizable” as baselines.

Check Jensen’s Inequality: To prove convexity, show for any $\mathbf{z}_1, \mathbf{z}_2$ and $\lambda \in [0,1]$:

$$L(\lambda \mathbf{z}_1 + (1 - \lambda) \mathbf{z}_2) \leq \lambda L(\mathbf{z}_1) + (1 - \lambda) L(\mathbf{z}_2)$$

If L is twice differentiable, check if the Hessian matrix $H = \nabla^2 L$ is positive semi-definite (all eigenvalues ≥ 0). Compute H symbolically or numerically. For non-differentiable (e.g., with abs or max), use subgradients or test with convex combination points.

The Hessian matrix H (or $\nabla^2 L$) is the square matrix of second-order partial derivatives of a scalar-valued loss function L with respect to its input variables (usually model parameters $\theta = [\theta_1, \theta_2, \dots, \theta_m]$, where m is the number of parameters). Mathematical Definition: For a twice-differentiable function $L : \mathbb{R}^m \rightarrow \mathbb{R}$, the Hessian is:

$$H_{ij} = \frac{\partial^2 L}{\partial \theta_i \partial \theta_j}, \quad i, j = 1, \dots, m$$

If H is positive semi-definite (all eigenvalues ≥ 0), L is convex, ensuring a global minimum (useful for convergence proofs).

Example: For a simple loss $L(\theta) = \theta_1^2 + 2\theta_2$, $H = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$

Example for MSE: $L(\mathbf{z}) = \|\mathbf{y} - X\mathbf{z}\|^2$ is quadratic, Hessian $2X^T X$ is positive semi-definite (since $X^T X$ has non-negative eigenvalues).

4.4.2 Derivatives

Compute Gradients; Check Lipschitz Constant $|\nabla L(\mathbf{z}_1) - \nabla L(\mathbf{z}_2)| \leq K \|\mathbf{z}_1 - \mathbf{z}_2\|$

Purpose: Gradients drive optimization (e.g., SGD); analyzing them ensures smooth, stable training. Lipschitz continuity bounds gradient changes, preventing exploding/vanishing gradients and ensuring convergence rates (e.g., in gradient descent, step sizes can be chosen safely).

Compute the Gradient: $\nabla L =$ partial derivatives w.r.t. each parameter in $\mathbf{z}$.

if $L = \frac{1}{n} \sum l(y_i, \hat{y}_i)$, then $\nabla L = \frac{1}{n} \sum \frac{\partial l}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial \mathbf{z}}$ (chain rule for NNs)

Example for MSE: $\nabla L = -2(y - \hat{y}) \cdot \frac{\partial \hat{y}}{\partial \mathbf{z}}$ (linear, bounded if data scaled).

Check Differentiability: Ensure L is differentiable everywhere (or almost; e.g., MAE not at 0).

Verify Lipschitz Continuity: Find constant K such that gradient difference is bounded by K times parameter difference. Analytical method: For convex loss function L , if gradients are bounded (e.g., for bounded data), it's L -Lipschitz.

5 Empirical Analysis

To empirically evaluate the performance of the proposed loss functions, we design a comprehensive asset pricing experiment. This section details the data, the construction of predictor variables, the backtesting procedure, and the evaluation metrics used to compare the models.

5.1 Data

The goal is to acquire monthly stock data from 1990 to 2025 to predict one-month-ahead returns.

1. WRDS Account Registration:

- Navigate to the WRDS website: <https://wrds-www.wharton.upenn.edu/>.
- Register for an account using your official XJTLU university email. Be sure to select "XJTLU" from the institutional drop-down menu.
- Account confirmation typically takes 1-2 business days.
- Important: You may receive an email from the university library asking for the research rationale. Please reply promptly, stating it is for academic research, and CC me on your response.

2. Data Extraction Procedure:

- Once logged in, find the Center for Research in Security Prices, LLC (CRSP) dataset under the "Subscriptions" heading on the homepage.
- Navigate through the following menus: "CRSP" → "Stock / Security Files" → "Monthly Stock File".

3. Query Form Configuration:

What we need to do is to use the features extracted from the monthly data to predict the stock return one month ahead. Works in finance literature typically predict returns at monthly or annual interval, such as Han (2021); Gu et al. (2020, 2021).

When you enter the "CRSP Monthly Stock" page, you may see a lot of options. Here is some suggestions for selecting data.

- “Step 1: Choose your date range”: Date Range: To avoid download errors from excessively large files, download the data in smaller increments (e.g., 1-2 years at a time). The complete required range is 1990 to 2025.
- “Step 2: Apply your company codes”: Select the option to “Search the entire database”
- “Step 3: Choose query variables”: Select the following essential variables: “(ret) Returns”, “(vol) Volume”, “(shrout) Number of Shares Outstanding”. Do not “select all” to keep the file size manageable. You can review variable definitions by clicking the question mark icon next to each.
- “Step 4: Select query output”: Select the output format as CSV (.csv) and choose Uncompressed. You may select your preferred date format.
- “Step 5: Download”: Click “Submit Form” and wait for the data file to be generated. Download the file and repeat the process until you have collected data for the entire 1990-2025 period.

Overall, we need at least 35 years data for empirical analysis. The data can cover the time periods from 1990 to 2025, which cover most of the well-known economic booming and recession periods.

5.2 Model Input and Output

The objective of our models is to predict the one-month-ahead stock return, $r_{i,t+1}$, for each stock i . This aligns with the standard asset pricing literature on return predictability Gu et al. (2020); Han (2021). The predictive relationship is modeled as:

$$r_{i,t+1} = f(\mathbf{X}_{i,t}) + \epsilon_{i,t+1}$$

where $f(\cdot)$ is the predictive model and $\mathbf{X}_{i,t}$ is the set of predictor variables (features) for stock i at time t . We construct three distinct sets of features, which are tested independently to ensure a robust evaluation.

Feature Set 1 ($\mathbf{X}_{i,t}^1$): We first define the input set $\mathbf{X}_{i,t}$. We consider three sets of input variables. Following Medhat and Schmeling (2021), we define our first set of variables, $\mathbf{X}_{i,t}^1$, as:

$$\begin{aligned} \text{cr}_{i,m} &= \prod_{j=t-m}^{t-1} (r_{i,j} + 1) - 1, \quad m = 1, 3, 6, 9, 12 \\ r_{i,t} &= \frac{p_{i,t}}{p_{i,t-1}} - 1 \\ \text{co}_{i,m} &= \sum_{j=t-m}^{t-1} (\text{to}_{i,j}), \quad m = 1, 3, 6, 9, 12 \\ \text{to}_{i,t} &= \frac{\text{vol}_{i,t}}{\text{os}_{i,t}} \end{aligned} \tag{8}$$

For stock i at time t , these variables are defined as follows: $r_{i,t}$ represents the simple return, whilst $\text{vol}_{i,t}$ denotes the total trading volume from $t-1$ to t . The variable $\text{os}_{i,t}$ represents the outstanding shares, and $\text{to}_{i,t}$ denotes the turnover. Additionally, $\text{cr}_{i,m}$ and $\text{co}_{i,m}$ represent the cumulative return and cumulative turnover, respectively, calculated over the past m months, where m takes values of 1, 3, 6, 9, and 12 months.

Feature Set 2 ($\mathbf{X}_{i,t}^2$): Drawing from Daniel and Moskowitz (2016); Han (2021), our second set of variables $\mathbf{X}_{i,t}^2$ is defined as:

$$\begin{aligned} \text{cr}_{i,m}^- &= \prod_{j=t-m}^{t-2} (r_{i,j} + 1) - 1, \quad m = 3, 6, 9, 12 \\ \text{Ncr}_{i,m} &= \frac{\text{cr}_{i,m}^- - \text{Avg cr}_m^-}{\text{Std cr}_m^-} \end{aligned} \tag{9}$$

where $\mathbf{X}_{i,t}^2$ consists of the normalised cumulative return ($\text{Ncr}_{i,m}$) calculated over m months, where m takes values of 3, 6, 9, and 12. This measure excludes the most recent month's return. Avg cr_m^- represents the cross-sectional average of $\text{cr}_{i,m}^-$, whilst Std cr_m^- denotes its cross-sectional standard deviation.

Feature Set 3 ($\mathbf{X}_{i,t}^3$): Our third set of input variables, $\mathbf{X}_{i,t}^3$, follows Daniel and Moskowitz (2016); Han (2021) and incorporates normalised monthly returns ($\text{nr}_{i,j}$) for the preceding 12 months. These returns are defined as:

$$\text{nr}_{i,j} = \frac{r_{i,j} - \text{Avg } r_j}{\text{Std } r_{i,j}}, \quad j = t-m, m = 1, \dots, 12 \tag{10}$$

where $\text{Avg } r_j$ represents the cross-sectional average of raw returns, and $\text{Std } r_{i,j}$ denotes their cross-sectional standard deviation. The normalisation is computed for each month j , ranging from $t - 1$ to $t - 12$. This set consists of 12 features. Table 1 summarizes the feature sets.

Table 1: This table summarises the variables in three input sets.

Input Sets	dimensions	past months	Input Variables
Set1 $\mathbf{X}_{i,t}^1$	$5 \times 2 = 10$	$m=1,3,6,9,12$	$\text{cr}_{i,m}, \text{co}_{i,m}$
Set2 $\mathbf{X}_{i,t}^2$	$4 \times 2 = 8$	$m=3,6,9,12$	$\text{Ncr}_{i,m}, \text{Avg cr}_m^-$
Set3 $\mathbf{X}_{i,t}^3$	$12 \times 2 = 24$	$j=t-m, m=1,2, \dots, 12$	$\text{nr}_{i,j}, \text{Avg } r_j$

5.3 Training Methodology and Portfolio Strategy

We employ a rolling-window backtesting procedure to simulate a realistic trading environment and mitigate look-ahead bias. The process is as follows:

1. **Initial Training:** The first model is trained on an initial 5-year data window (e.g., Jan 1990 - Dec 1994).
2. **Prediction Period:** The trained model is used to generate one-month-ahead return predictions for the subsequent 6 months (e.g., Jan 1995 - Jun 1995).
3. **Re-training:** The training window is rolled forward by 6 months (e.g., the new window becomes Jul 1990 - Jun 1995). A new model is trained on this updated dataset.

This cycle of predicting for 6 months and re-training is repeated until the end of our sample period.

For each month in the prediction period, we form a long-short portfolio based on the model's return forecasts. All stocks are sorted in descending order of their predicted returns. We take a long position in the top decile (top 10%) and a short position in the bottom decile (bottom 10%). The performance of these equal-weighted portfolios is then tracked over time.

5.4 Models

5.4.1 Basic Models

We evaluate a suite of nine models, all based on a feed-forward neural network (NN) architecture but differing in their loss function.

- **Baseline Models:** We establish a benchmark using standard loss functions: Mean Squared Error (NN_{Mean}) and Median Squared Error ($\text{NN}_{\text{Median}}$). The hyperparameters (e.g., number of layers, neurons) for a simple NN are tuned via grid search for the NN_{Mean} model, and this fixed architecture is then used for all subsequent models to ensure a fair comparison of the loss functions.
- **MADL-based Models:** We implement models using the Mean Absolute Deviation Loss (NN_{MADL}) and its generalized version (NN_{GMADL}), following the work of Michańków et al. (2024b,a).
- **Proposed Loss Function Models:** We implement NNs with each of our proposed loss functions from Sections 3.2.1 to 3.2.5, denoted as NN_{Flip} , NN_{Huber} , $\text{NN}_{\text{Quantile}}$, NN_{Bound} , and $\text{NN}_{\text{Ranking}}$.

5.5 Performance Evaluation Metrics

To provide a thorough comparison, we assess both the predictive accuracy and the economic value of the models using the following metrics:

- **Predictive Accuracy:** We calculate the out-of-sample Root Mean Squared Error (RMSE) for the entire test period. We also plot the rolling 12-month RMSE to analyze how predictive performance evolves over time.
- **Portfolio Performance:** We compute and plot the cumulative returns of the long, short, and long-short portfolios.
- **Risk-Adjusted Returns:** We calculate the overall Sharpe Ratio for the long-short portfolio over the entire test period. Additionally, we plot the rolling 12-month Sharpe Ratio to assess the stability of risk-adjusted performance.

Research To-Do List

Phase 1: Data Acquisition

1. Register for WRDS Access:

- Go to the WRDS website: <https://wrds-www.wharton.upenn.edu/>.

- Register for an account using your official university email (@xjtlu.edu.cn). Select “Xi’an Jiaotong-Liverpool University” from the institution list.
- Wait for approval (1-2 business days). You may receive an email from the university library asking for the research rationale. Reply promptly, explaining the project is for academic research, and CC me on the email.

2. Download CRSP Monthly Data:

- Once logged in, navigate to CRSP under “Subscriptions”.
- Select “Stock / Security Files” → “Monthly Stock File”.
- **Step 1 (Date Range):** Download the data in manageable chunks (e.g., 5-10 years at a time) to avoid timeouts. The full range is **Jan 1990 to Dec 2025**.
- **Step 2 (Company Codes):** Select “Search the entire database”.
- **Step 3 (Query Variables):** Select at a minimum: PERMNO, DATE, RET (Returns), VOL (Volume), and SHROUT (Shares Outstanding). Familiarize yourself with the definitions by clicking the question mark next to each variable.
- **Step 4 (Query Output):** Choose output format as **CSV (.csv)** and compression as **Uncompressed**.
- Submit the query and download the data files. Consolidate them into a single master data file.

Phase 2: Data Preprocessing & Feature Engineering

1. **Clean Data:** Handle missing values for RET, VOL, and SHROUT.
2. **Calculate Base Variables:** Compute simple returns ($r_{i,t}$) and turnover ($to_{i,t}$).
3. **Engineer Feature Sets:** Write scripts to generate the three feature sets ($\mathbf{X}^1, \mathbf{X}^2, \mathbf{X}^3$) for every stock-month observation as defined in the methodology. This will involve significant data manipulation (e.g., rolling products, cross-sectional normalization).
4. **Define Target Variable:** The target variable is the one-month-ahead return, $r_{i,t+1}$.

Phase 3: Model Implementation & Training

1. Setup Baseline NN:

- Implement a standard feed-forward neural network in Python (using TensorFlow or PyTorch).
- Perform a grid search for the optimal hyperparameters (e.g., layers, neurons, learning rate, activation function) using the NN_{Mean} model and one of the feature sets (e.g., $\mathbf{X}^1$).
- **Crucially, fix this architecture.** This exact structure will be used for all other models to ensure we are comparing the loss functions, not the architectures.

2. Implement All Loss Functions:

- Code the custom loss functions for $\text{NN}_{\text{Median}}$, NN_{MADL} , NN_{GMADL} , NN_{Flip} , NN_{Huber} , $\text{NN}_{\text{Quantile}}$, NN_{Bound} , and $\text{NN}_{\text{Ranking}}$.

3. Implement Backtesting Loop:

- Create the rolling-window training script as described in Section 5.3 (5-year train, 6-month test, 6-month slide).
- This loop will iterate through time, re-training each of the nine models at every step and storing the out-of-sample predictions.

Phase 4: Analysis & Results Generation

1. **Run Full Backtest:** Execute the backtesting script for all nine models on all three feature sets. This will be computationally intensive.
2. **Calculate Performance Metrics:** For each model and feature set combination:
 - Calculate and tabulate the overall out-of-sample RMSE.
 - Generate data for the rolling 12-month RMSE plots.
 - Calculate the monthly returns for the long, short, and long-short decile portfolios.
 - Generate data for the cumulative return plots.
 - Calculate and tabulate the overall Sharpe Ratio for the long-short portfolio.
 - Generate data for the rolling 12-month Sharpe Ratio plots.

3. **Visualize Results:** Create high-quality plots for the paper (cumulative returns, rolling RMSE, rolling Sharpe Ratios).
4. **Summarize Findings:** Create tables comparing the key metrics (overall RMSE, overall Sharpe Ratio) across all models and feature sets.

6 Appendix

References

- Daniel, K., Moskowitz, T.J., 2016. Momentum crashes. *Journal of Financial economics* 122, 221–247.
- Gu, S., Kelly, B., Xiu, D., 2020. Empirical asset pricing via machine learning. *The Review of Financial Studies* 33, 2223–2273.
- Gu, S., Kelly, B., Xiu, D., 2021. Autoencoder asset pricing models. *Journal of Econometrics* 222, 429–450.
- Han, C., 2021. Bimodal characteristic returns and predictability enhancement via machine learning. *Management Science* .
- Medhat, M., Schmeling, M., 2021. Short-term momentum. *The Review of Financial Studies* 35, 1480–1526.
- Michańków, J., Sakowski, P., Ślepaczuk, R., 2024a. Generalized mean absolute directional loss as a solution to overfitting and high transaction costs in machine learning models used in high-frequency algorithmic investment strategies. *arXiv preprint arXiv:2412.18405* .
- Michańków, J., Sakowski, P., Ślepaczuk, R., 2024b. Mean absolute directional loss as a new loss function for machine learning problems in algorithmic investment strategies. *Journal of Computational Science* 81, 102375.
- Vapnik, V.N., 1999. An overview of statistical learning theory. *IEEE transactions on neural networks* 10, 988–999.